



Desenvolvimento de Software

Prof. Vinicius Cardoso Garcia
vcg@cin.ufpe.br :: @vinicius3w :: assertlab.com
http://viniciusgarcia.me

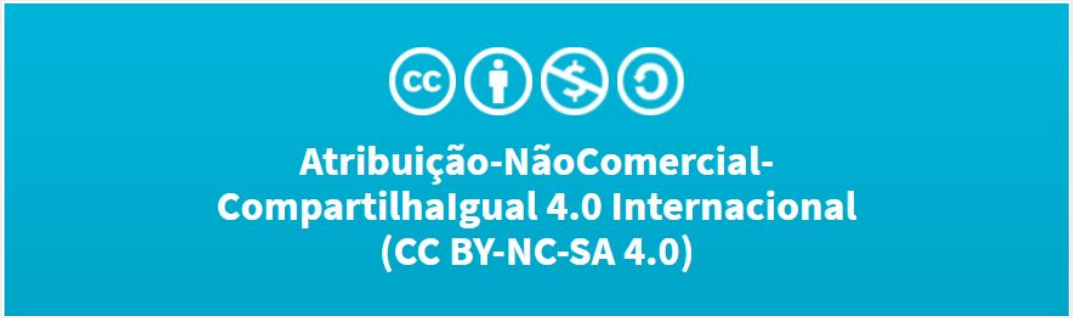




Universidade
Federal
de Pernambuco

Licença do material

Este Trabalho foi licenciado com uma Licença



Mais informações visite

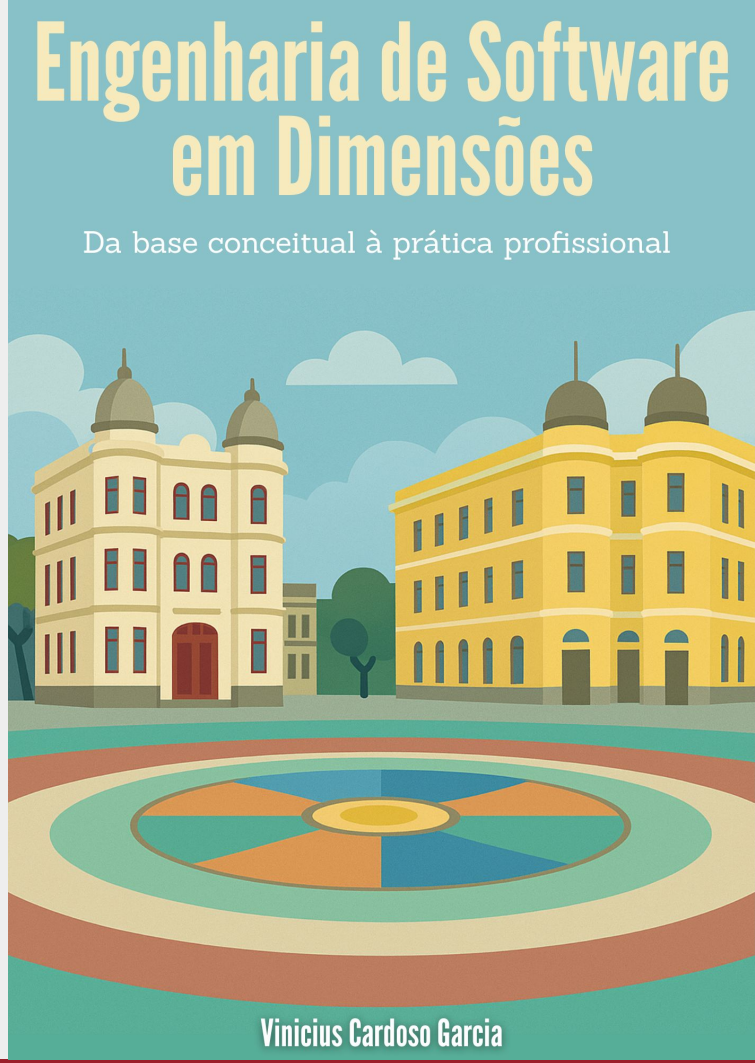
https://creativecommons.org/licenses/by-nc-sa/4.0/deed.pt_BR

Engenharia de Software em Dimensões

É um material abrangente, didático e atualizado que apresenta a Engenharia de Software como uma disciplina multidimensional. Cada capítulo foca em uma dimensão prática, conectando teoria e realidade profissional com clareza, profundidade e propósito.

Prof. Vinicius Cardoso Garcia

https://bit.ly/esdbook



Ponto de Commit

O podcast figital da disciplina (IF977) Engenharia de Software do curso de bacharelado em Sistemas de Informação do Centro de Informática (CIn) da Universidade Federal de Pernambuco (UFPE).

https://go.uaite.xyz/ponto-de-commit



Engenharia de Software Moderna

Engenharia de Software Moderna é um livro destinado a alunos de cursos de graduação em Computação. Ele pode ser lido também por profissionais que buscam um conhecimento horizontal da área.

Prof. Marco Tulio Valente

https://engsoftmoderna.info

@engsoftmoderna



Do Código ao Container

Entenda o problema real que o Docker resolve, aprenda os conceitos fundamentais e comece a escrever Dockerfiles para backends e frontends modernos.



O Que Vamos Aprender Hoje

01

O Problema

Por que "na minha máquina funciona" é um risco técnico real

02

Conceitos Fundamentais

Imagem vs. container — qual a diferença e por que importa

03

Anatomia do Dockerfile

As instruções essenciais para descrever seu ambiente

04

Dockerfiles Reais

Backend Node.js/Express e frontend React com multi-stage build

05

Ciclo de Vida Básico

Build, run, ps, logs e stop na prática

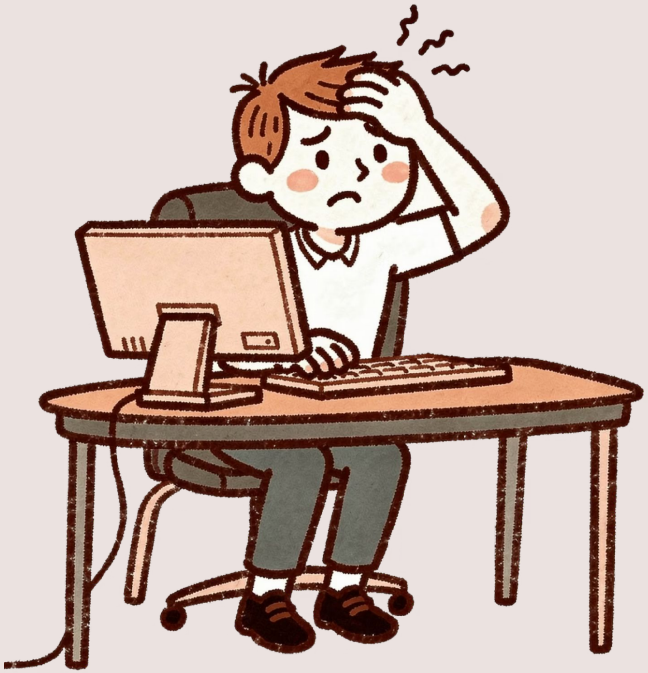


Antes de qualquer conceito

"Alguém do time tentou rodar o projeto de outro colega esse semestre e algo quebrou. O que foi?"

Esse problema tem nome. E tem solução.





"Na Minha Máquina Funciona"

Essa frase é uma das mais temidas no desenvolvimento de software. O código roda perfeitamente no seu notebook, mas quebra em produção, ou na máquina de outro desenvolvedor da equipe. Por que isso acontece?

Versões Diferentes

Node 22 no dev, Node 24 em produção: dependências podem quebrar sem aviso.

Variáveis de Ambiente

Configurações locais que nunca foram documentadas: portas, credenciais, caminhos de sistema.

Sistema Operacional

Windows, macOS e Linux têm comportamentos distintos em caminhos, permissões e binários nativos.

A Raiz Técnica do Problema

Ambiente Implícito

Toda aplicação depende de um conjunto de software ao seu redor: runtime, bibliotecas do SO, variáveis de configuração e serviços externos. Esse ambiente raramente é descrito de forma explícita e versionada, ele simplesmente existe na máquina de quem desenvolveu.

O Resultado Prático

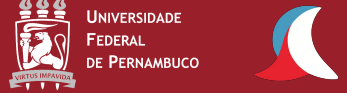
Bugs que aparecem apenas em produção, onboarding lento de novos devs (horas configurando o ambiente), pipelines de CI que se comportam de forma diferente do local, e deploys imprevisíveis.

⚠ A solução não é documentar melhor o README, é tornar o ambiente parte do código.

Docker empacota o ambiente junto com o código.

Não apenas as dependências do `package.json`, mas o runtime, o sistema de arquivos e as configurações necessárias para rodar a aplicação em qualquer lugar.

Centro de Informática UFPE
Universidade Federal de Pernambuco



Docker empacota código + ambiente juntos

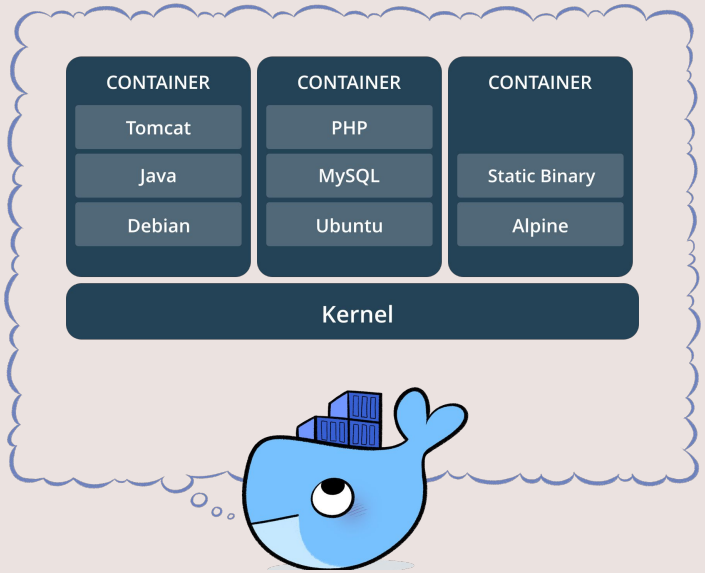


Uma imagem Docker é imutável, portável e versionável — como um artefato de software que carrega tudo o que precisa para rodar. A partir de uma única imagem, você pode gerar dezenas de containers idênticos rodando em qualquer ambiente: máquina do dev, CI/CD, staging ou produção.

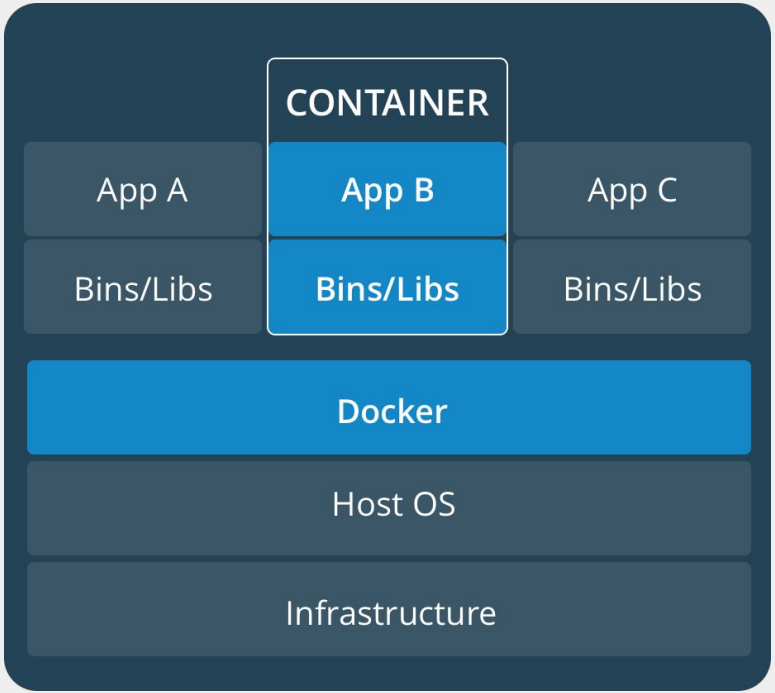
Imagem	Container
O pacote (a receita): imutável, portável, versionável. Construída via Dockerfile.	A instância em execução (o bolo assado a partir da receita). Tem processo, memória e estado.

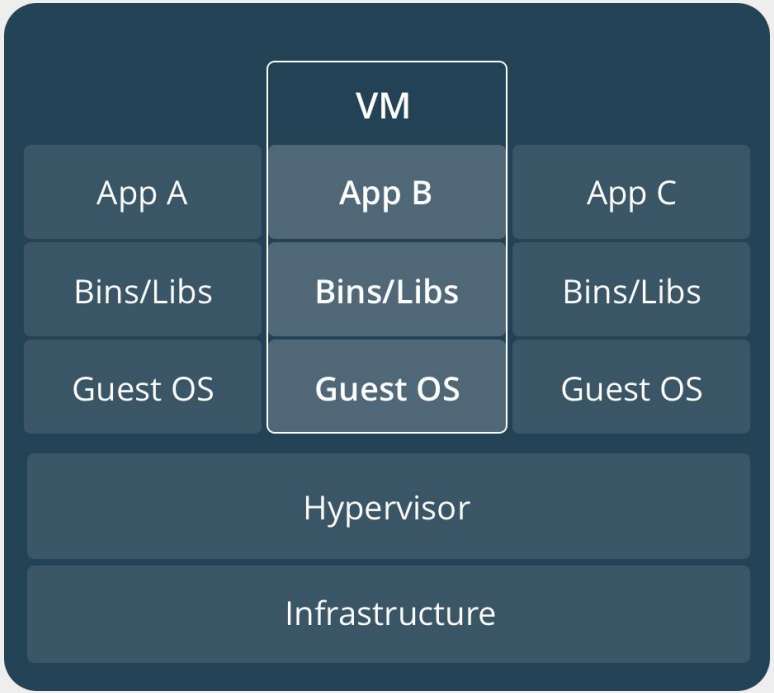
O que são contêineres?

- Contêineres são processos isolados rodando sobre o sistema operacional hospedeiro.
  - Namespaces: limitam o que o processo vê (arquivos, rede, processos, usuários).
  - cgroups: limitam quanto ele usa (CPU, memória, I/O).
  - Capabilities e filtros de syscalls restringem privilégios.
- Todos compartilham o kernel do host: não há SO convidado, por isso sobem em segundos.
- No Windows e no macOS, o Docker Desktop roda uma VM Linux para fornecer esse kernel.

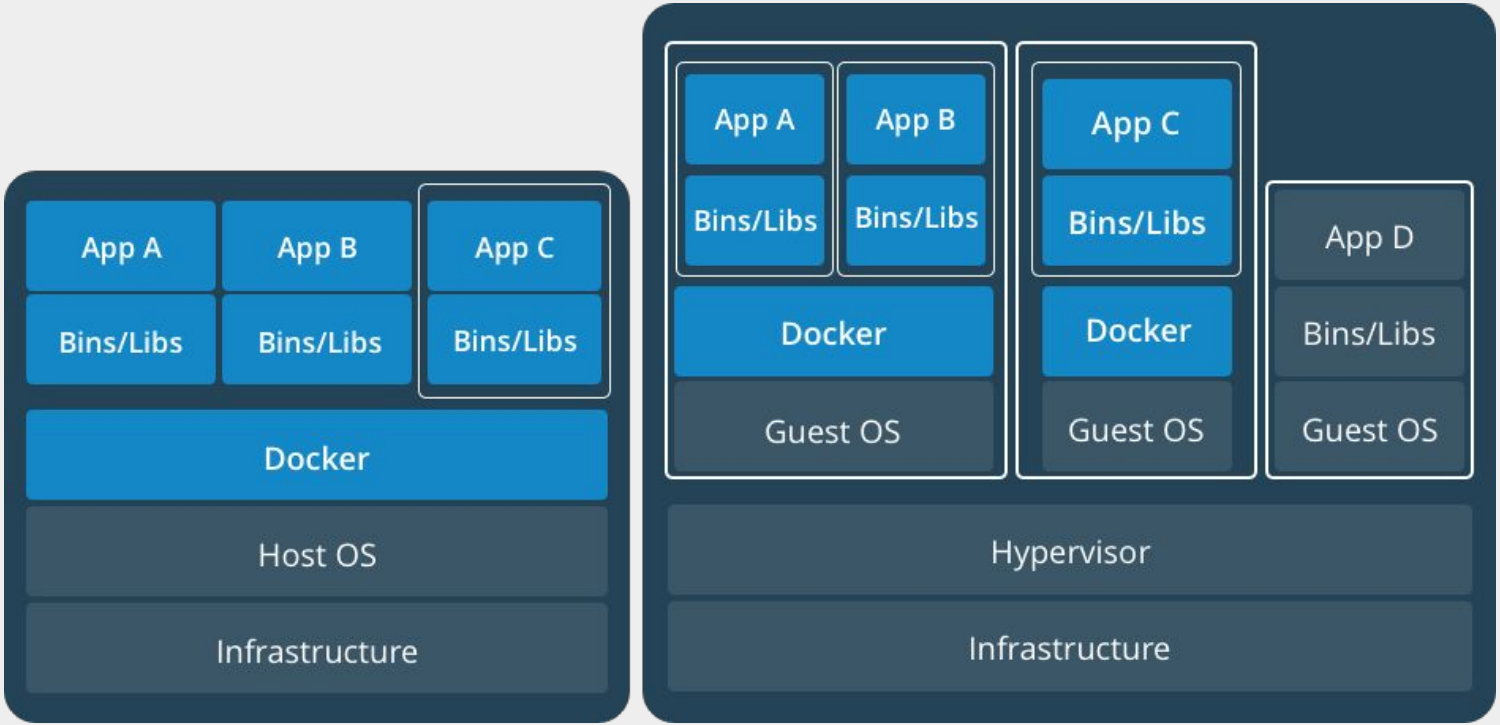


A diferença entre máquinas virtuais e contêineres





Contêineres e máquinas virtuais juntos



A distinção mais importante

Compreender a diferença entre imagem e container é o primeiro passo para trabalhar com Docker com segurança e eficiência.

	Imagem	Container
Analogia	Receita de bolo	O bolo assado
Estado	Imutável	Tem processo, memória, estado
Origem	Construída via Dockerfile	Instanciada a partir de uma imagem
Ciclo de vida	Existe até docker rmi	Parado até docker rm

Você pode ter uma imagem e zero containers, ou uma imagem e cem containers rodando simultaneamente, todos idênticos.

O Dockerfile: A Receita do Ambiente

O `Dockerfile` é um arquivo de texto que descreve, passo a passo, como construir uma imagem Docker. Cada instrução gera uma **camada** no sistema de arquivos da imagem. Essas camadas são cacheadas pelo Docker para acelerar rebuilds.



Ao versionarmos o Dockerfile junto com o código-fonte, garantimos que qualquer desenvolvedor ou servidor de CI possa reproduzir exatamente o mesmo ambiente, eliminando a dependência de configurações manuais.

Dockerfile — a receita escrita em texto

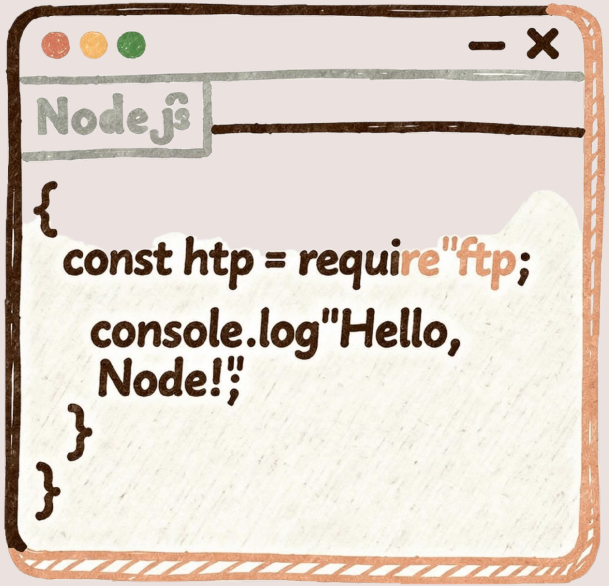
O Dockerfile é um arquivo de texto com instruções que o Docker executa sequencialmente para construir uma imagem. Cada instrução gera uma camada no sistema de arquivos da imagem.

Instrução	O que faz	Exemplo
`FROM`	Imagem base de partida	`FROM node:24-alpine`
`WORKDIR`	Diretório de trabalho no container	`WORKDIR /app`
`COPY`	Traz arquivos do host para o container	`COPY . .`
`RUN`	Executa comando durante o build	`RUN npm ci`
`EXPOSE`	Documenta a porta (não publica)	`EXPOSE 3000`
`CMD`	Comando padrão ao iniciar o container	`CMD ["node", "src/index.js"]`



Dockerfiles na Prática

Vamos analisar Dockerfiles reais do projeto **PetFood**, um backend em Node.js/Express e um frontend em React. Prestaremos atenção especial a estratégias de otimização que fazem diferença no dia a dia



Backend PetFood — Node.js/Express containerizado

```
# Imagem base leve (Node 24 LTS)
FROM node:24-alpine

# Diretório de trabalho
WORKDIR /app

# Copia só o manifesto primeiro
# — aproveita cache de camadas
COPY package*.json ./
RUN npm ci --omit=dev

# Código-fonte separado para não
# invalidar cache de dependências
COPY . .

EXPOSE 3000
CMD ["node", "src/index.js"]
```

💡 Por que copiar package*.json antes?

Se só o código mudar, o Docker reaproveita do cache a camada do npm ci, e o build não reinstala as dependências.

Essa ordem é uma das otimizações mais importantes que você pode aplicar em qualquer Dockerfile Node.js.

Por Que a Ordem Importa?

❌ Ordem Ingênua

```
COPY . .
RUN npm ci
```

Qualquer mudança no código (até um comentário) invalida o cache do npm ci: as dependências são baixadas de novo a cada build.

✅ Ordem Otimizada

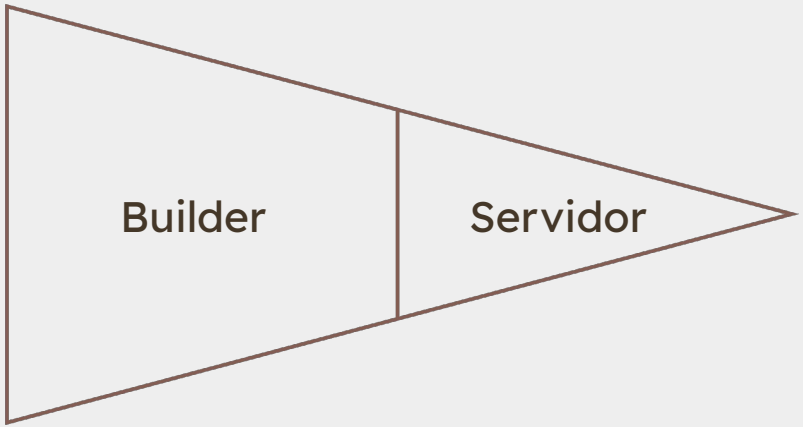
```
COPY package*.json ./
RUN npm ci
COPY . .
```

O cache do `npm ci` só é invalidado quando `package.json` muda. Mudanças no código-fonte não reexecutam a instalação de dependências.



Multi-Stage Build para o Frontend React

O React precisa de um passo de compilação (`npm run build`) que gera arquivos estáticos. Mas você não quer carregar o Node.js e todas as `node_modules` em produção. A solução é o **multi-stage build**.



O resultado é uma imagem de produção enxuta: apenas o nginx e os arquivos estáticos, sem Node.js, sem código-fonte, sem node_modules. A imagem final fica muito menor do que uma imagem ingênua.

Frontend PetFood — React precisa de uma estratégia diferente

⚠ React precisa de Node para compilar, mas em produção só precisamos dos arquivos estáticos gerados pelo build.

```
# Stage 1: compilar o React com Node
FROM node:24-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
ARG VITE_API_URL=/api       # o Vite embute este valor no build
RUN npm run build          # gera /app/dist
# Stage 2: servir os arquivos estáticos com nginx
FROM nginx:alpine
COPY nginx.conf /etc/nginx/conf.d/default.conf
COPY --from=build /app/dist /usr/share/nginx/html
EXPOSE 80
```

Stage 1 — descartado

Node + npm + código-fonte: usado apenas para compilar, não vai para a imagem final.

Imagem final — nginx

Apenas nginx, sua configuração e os arquivos estáticos: algumas dezenas de MB (confira com docker images).



nginx.conf — o elo com o Compose

```
# nginx.conf (na pasta do frontend)
server {
  listen 80;
  root /usr/share/nginx/html;

  # SPA: rotas do React Router
  # caem no index.html
  location / {
    try_files $uri $uri/ /index.html;
  }

  # Chamadas à API vão para o backend
  location /api/ {
    proxy_pass http://backend:3000;
  }
}
```

💡 Por que um nginx.conf?

O nginx padrão só serve arquivos. Esta configuração resolve dois problemas: recarregar uma rota do React sem 404 e encaminhar /api para o backend.

O nome backend só existe na rede do Docker Compose. Rodando este container sozinho, o nginx não encontra esse host e não sobe: é o gancho para a aula de amanhã.



Benefícios do Multi-Stage Build

🏋️ Imagem Menor

A imagem final não inclui Node.js, npm ou `node_modules`. Apenas o nginx e os arquivos HTML, CSS e JS compilados.

🔒 Superfície de Ataque Reduzida

Menos ferramentas na imagem significa menos vetores de vulnerabilidade. O container de produção não tem compiladores nem gerenciadores de pacotes.

⚡ Deploy Mais Rápido

Imagens menores são transferidas mais rápido entre registries e servidores, acelerando pipelines de CI/CD e deploys.



O Arquivo .dockerignore

O Que É?

Funciona como o `.gitignore`: lista arquivos e pastas que devem ser **excluídos do contexto de build**. O Docker envia todo o contexto para o daemon antes de processar o Dockerfile.

Exemplo Típico

```
node_modules
.git
.env
*.log
dist
coverage
.DS_Store
```

Por Que É Importante?

Performance: Sem `.dockerignore`, a pasta `node_modules` inteira é enviada ao daemon antes do build, podendo ter centenas de MB desnecessários.

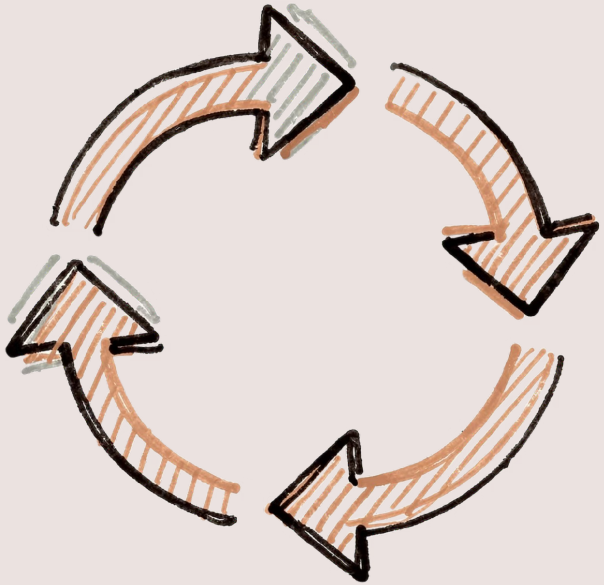
Segurança: Evita que arquivos `.env` com credenciais sejam acidentalmente copiados para dentro da imagem.

Correção: Garante que o `node_modules` local não sobrescreva as dependências instaladas dentro do container.



Ciclo de Vida Básico de um Container

Com o Dockerfile pronto, é hora de aprender os comandos essenciais para construir, executar e inspecionar containers no dia a dia. Esses são os comandos que você usará em todo projeto Docker.



docker build: Construindo a Imagem

```
# Construir uma imagem com tag
docker build -t petfood-backend:1.0 .

# O ponto (.) indica o contexto de build (diretório atual)
# -t define o nome e a tag da imagem
```

O que acontece por baixo

O Docker lê o `Dockerfile`, processa cada instrução sequencialmente, verifica o cache para cada camada e gera uma imagem identificada por um hash SHA256 único.

Tags e versionamento

Use tags semânticas como `:1.0`, `:latest` ou `:production`. A tag `latest` é o padrão quando nenhuma é especificada, mas evite depender dela em produção.



docker run: Iniciando um Container

```
# Modo básico
docker run petfood-backend:1.0

# Com publicação de porta: host:container
docker run -p 3000:3000 petfood-backend:1.0

# Em segundo plano (detached) com nome personalizado
docker run -d --name backend -p 3000:3000 petfood-backend:1.0

# Passando variáveis de ambiente
docker run -d -p 3000:3000 -e NODE_ENV=production petfood-backend:1.0
```

A flag `-p 3000:3000` mapeia a porta 3000 do host para a porta 3000 do container. Sem esse mapeamento, a aplicação roda isolada e inacessível externamente.



Inspecionando e Gerenciando Containers

`docker ps`

Lista containers em execução. Use `docker ps -a` para ver todos, incluindo os parados. Mostra ID, imagem, status, portas e nome.

`docker logs`

Exibe os logs de saída (stdout/stderr) de um container. Use `docker logs -f backend` para acompanhar em tempo real (follow mode).

`docker stop`

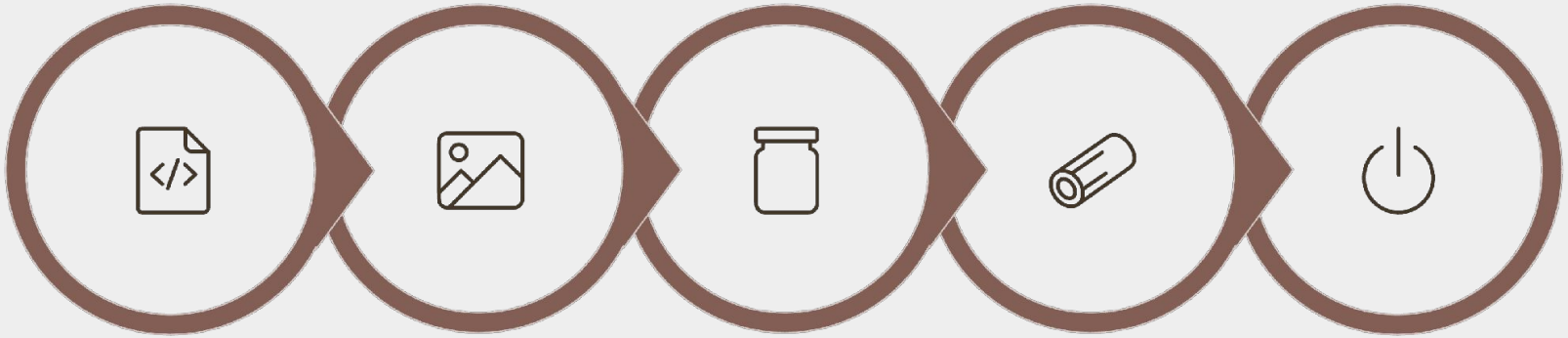
Para um container graciosamente, enviando sinal SIGTERM e aguardando 10 segundos antes de forçar com SIGKILL. Use `docker stop backend`.

`docker rm`

Remove um container parado. Use `docker rm backend`. Para remover automaticamente ao parar, adicione `--rm` ao `docker run`.



Fluxo Completo: Do Dockerfile ao Container Rodando



Esse fluxo se repete em todo projeto. No Podman os comandos são os mesmos (podman build, podman run), e ele aceita Containerfile no lugar de Dockerfile.

Comparativo: Antes e Depois do Docker

Situação	Sem Docker	Com Docker
Onboarding de novo dev	Horas instalando dependências e resolvendo conflitos	`docker run` e pronto em minutos
Paridade de ambiente	Diferenças entre dev, staging e produção	Mesma imagem em todos os ambientes
Versão do Node	Depende do que está instalado no SO	Definida no `FROM` do Dockerfile
Deploy	Scripts manuais, documentação desatualizada	Imagem versionada e reproduzível
Tamanho da imagem de produção (React)	Centenas de MB com Node.js	Dezenas de MB com nginx + multi-stage



node:24 vs. node:24-alpine — qual a diferença?

	node:24	node:24-alpine
Tamanho	~1 GB	~150 MB
Base	Debian (glibc)	Alpine (musl)
Uso recomendado	Build e ferramentas de dev	Produção, sem deps nativas

Imagem menor = menos superfície de ataque e deploy mais rápido. Mas o musl do Alpine pode quebrar dependências nativas (bcrypt, sharp, Prisma). Nesse caso, use `node:24-slim`.



Boas Práticas Resumidas

Use imagens enxutas

Prefira node:24-alpine (ou node:24-slim, com dependências nativas) a node:24.

Use npm ci

Em vez de `npm install`, use `npm ci` para builds reproduzíveis baseadas no `package-lock.json`.

Sempre use .dockerignore

Exclua `node_modules`, `.env` e `.git` para builds mais rápidos e seguros.

Otimize a ordem das camadas

Coloque o que muda menos primeiro para aproveitar o cache e acelerar rebuilds.

O que você mudaria se pudesse começar o semestre de novo com essa ferramenta?

"O Docker resolve o problema de ambiente. Mas esse problema existia no seu projeto esse semestre. O que sua equipe fez para lidar com ele sem Docker?"



Hoje: um container. Amanhã: um sistema inteiro.

O que vimos hoje

- O problema que Docker resolve e por que importa profissionalmente
- Imagem vs. container — a distinção fundamental
- Dockerfile do PetFood — backend Node.js e frontend React
- Ciclo de vida básico de um container

Amanhã — terça-feira

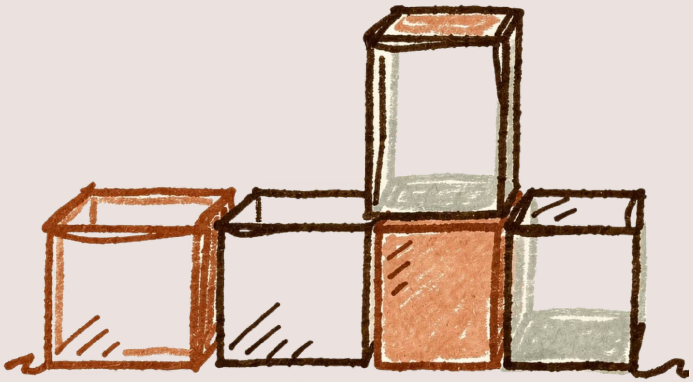
O PetFood tem três serviços: **frontend React**, **backend Node.js/Express** e **banco PostgreSQL**. Como subir os três juntos com um único comando?

Docker Compose

Essa é a resposta, e é o que veremos na próxima aula.



Referências e próximos passos



Documentação Oficial

docs.docker.com — referência completa de comandos, Dockerfile e boas práticas. Sempre atualizada e com guias de início rápido.

Pratique agora

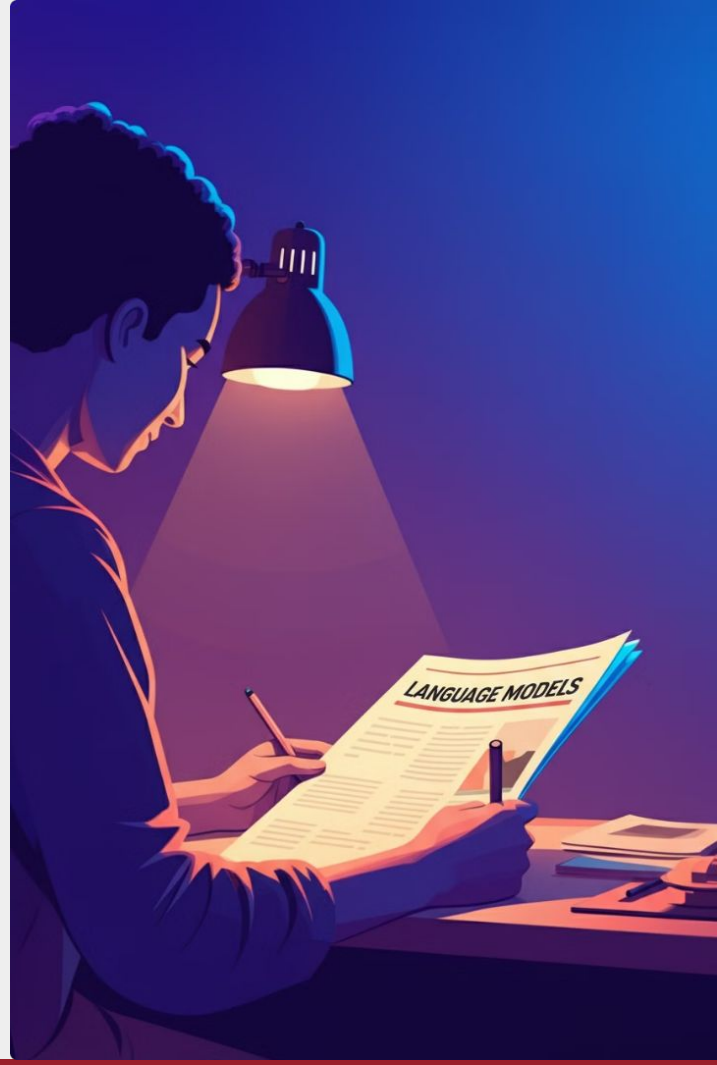
Clone o repositório do PetFood, escreva o Dockerfile do backend seguindo o modelo da aula e rode docker build + docker run localmente.

Docker Hub

hub.docker.com — repositório público de imagens. Explore as imagens oficiais do Node, nginx e PostgreSQL que usamos no PetFood.

Próximos passos

- Eng. Software em Dimensões
  - Cap. 10, seção 10.3.2 — Container scanning no pipeline (Trivy, Anchore, Clair)
  - Cap. 14, seção 14.5.2 — Plataformas de orquestração de contêineres
- Eng. Software Moderna
  - Cap. 10, seção 10.4 — Deployment Contínuo
- Documentação oficial
  - Dockerfile reference — docs.docker.com/reference/dockerfile
  - Node.js releases — nodejs.org/en/about/previous-releases



contacts

vcg@cin.ufpe.br
@vinicius3w
viniciusgarcia.me

academy

go.uaite.xyz/vcg-es
go.uaite.xyz/vcg-microservices
go.uaite.xyz/vcg-devops
go.uaite.xyz/vcg-esaia
go.uaite.xyz/vcg-deso

Centro de Informática UFPE
Universidade Federal de Pernambuco
ASSERT LAB

Desenvolvimento de Software (CIN0136)

Do Código ao Container